Tugas 3 - Sinkronisasi dan Distributed Systems

Mata Kuliah Sistem Paralel dan Terdistribusi

Dosen Pengampu:
Riska Kurniyanto Abdullah, S.T., M.Kom.

Penulis:
Naufal Tiarana Putra - 11231071

INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN
2026

DAFTAR ISI

BAB I **1**
PENDAHULUAN **1**
1.1. Tujuan 1
1.2. Batasan Sistem 1
BAB II **2**
SPESIFIKASI MASALAH DAN TARGET RUBRIK **2**
2.1. Requirement Project 2
2.2. Pemetaan Requirement terhadap Modul dan Bukti 2
BAB III **4**
DESAIN SISTEM **4**
3.1. Arsitektur 3-Node 4
3.2. Alur Konsensus Raft 5
3.3. Desain Lock, Queue dan Cache 5
3.4. Skema Deployment (Manual & Docker) 5
3.5. Failure Detection dan Metrics 7
BAB IV **8**
IMPLEMENTASI TIAP KOMPONEN **8**
4.1. Implementasi Lock Manager 8
4.2. Implementasi Distributed Queue 9
4.3. Implementasi Cache Coherence 10
4.4. API dan OpenAPI/Swagger 11
4.5. Failure Detector dan Metrics 11
BAB V **12**
SKENARIO UJI DAN HASIL FUNGSIONAL **12**
5.1. Skenario Uji Lock 12
5.2. Skenario Uji Queue 12
5.3. Skenario Uji Cache 13
5.4. Uji Integrasi 3 Node 14
5.5. Perbandingan Mode Uji 14
BAB VI **17**
ANALISIS PERFORMA DAN TRADE-OFF **17**
6.1. Metode Benchmark 17
6.2. Hasil Angka (rata-rata, p95, max) 17
6.3. Interpretasi 17
6.4. Dampak Konsistensi terhadap Throughput/Latency 18
BAB VII **19**
KESIMPULAN **19**

BAB I
PENDAHULUAN

1.1. Tujuan

Tujuan utama sistem adalah:

1. Membangun koordinasi antar tiga node menggunakan mekanisme leader election berbasis Raft.
2. Menyediakan distributed lock manager yang mendukung shared lock, exclusive lock, dan deteksi deadlock.
3. Menyediakan distributed queue yang tahan simpan melalui SQLite dan dapat direplikasi antar node.
4. Menyediakan cache coherence dengan invalidasi antar node, versioning, dan kebijakan LRU.
5. Menyediakan API HTTP yang terdokumentasi melalui OpenAPI/Swagger.
6. Menyediakan dua mode deployment, yaitu manual dan Docker Compose, agar sistem mudah direplikasi untuk demo dan pengujian.

1.2. Batasan Sistem

Implementasi ini sengaja dibatasi pada skenario tiga node untuk menjaga kesederhanaan arsitektur dan memudahkan verifikasi perilaku konsensus. Raft yang dipakai berfokus pada leader election dan heartbeat, bukan log replication penuh seperti implementasi Raft produksi.

Untuk queue, replikasi dilakukan secara best-effort dengan metadata ack dan retry, tetapi belum menggunakan quorum commit formal. Untuk cache, model coherence memakai state lokal bergaya MESI yang cukup untuk invalidasi dan konsistensi praktis, namun belum berupa direktori MESI penuh. Dengan demikian, sistem cocok untuk pembelajaran, demo, dan dasar pengembangan lanjutan, tetapi belum ditujukan sebagai layanan produksi skala besar.

BAB II
SPESIFIKASI MASALAH DAN TARGET RUBRIK

2.1. Requirement Project

Secara ringkas, requirement yang ditargetkan oleh proyek adalah:

1. Sistem dapat menjalankan lebih dari satu node dan memilih leader secara otomatis.
2. Sistem menyediakan distributed lock manager dengan operasi acquire, release, dan status.
3. Sistem menyediakan distributed queue dengan enqueue, dequeue, status, mekanisme persistensi, dan replikasi best-effort.
4. Sistem menyediakan cache coherence dengan set, get, delete, invalidate, serta kebijakan replacement.
5. Sistem menyediakan deteksi deadlock dan endpoint administrasi untuk resolusi.
6. Sistem memiliki dokumentasi API yang dapat diakses melalui Swagger/OpenAPI.
7. Sistem dapat dijalankan dalam mode lokal maupun Docker dengan konfigurasi `.env`.
8. Sistem dapat dibuktikan melalui unit test, integration test, dan benchmark.

2.2. Pemetaan Requirement terhadap Modul dan Bukti

Table 2.1 Pemetaan Requirement

Requirement	Modul Utama	Bukti Implementasi	Bukti Uji
Leader election dan koordinasi node	Raft node dan BaseNode	Raft election, heartbeat, forwarding request	Uji 3 node dan log leader election
Distributed lock manager	lock_manager dan endpoint lock	Shared/exclusive lock, wait queue, status lock	test_lock_admin, test_deadlock_detection
Deadlock detection	lock_manager	Graph wait-for dan resolusi victim	test_deadlock_detection
Distributed queue	queue_manager dan endpoint queue	Consistent hashing, SQLite persistence, replication metadata	test_queue_replication, integration 3 node
Cache coherence	cache_manager	State MESI-like,	test_cache_mesi,

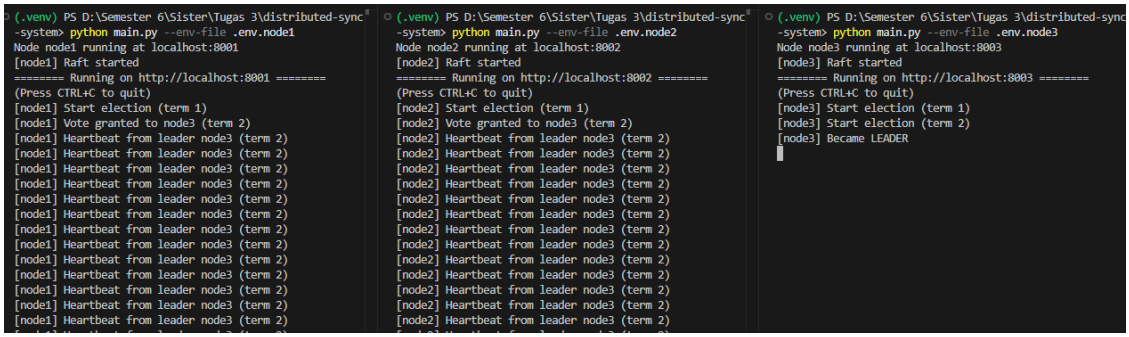
	dan endpoint cache	invalidasi, LRU	integration 3 node
API dokumentasi	api_spec.yaml dan docs	OpenAPI/Swagger endpoint list	test_openapi_docs
Failure detection dan observability	failure_detector dan metrics	Heartbeat tracking, suspected peer tracking, counters/gauges	smoke test health dan metrics
Deployment manual dan Docker	README, deployment guide, docker-compose	CLI env, .env, Compose multi-container	percobaan manual dan docker
Benchmark performa	benchmarks dan performance report	CSV hasil uji	benchmarks/results.csv

BAB III
DESAIN SISTEM

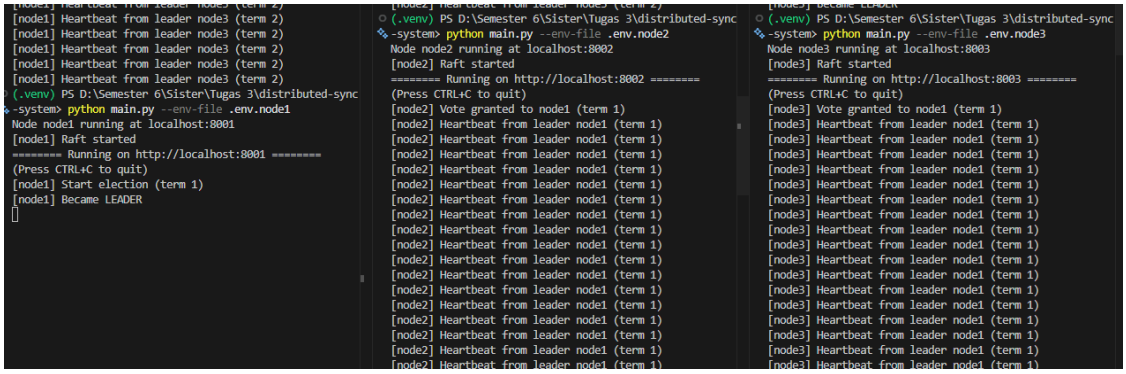
3.1. Arsitektur 3-Node

Arsitektur sistem terdiri dari tiga node yang masing-masing dapat menjalankan layanan HTTP, proses konsensus Raft, dan state lokal untuk lock, queue, dan cache. Setiap node memiliki identitas, port, daftar peer, dan informasi region yang dapat diatur melalui environment variable atau flag CLI.

Node menerima request dari client melalui endpoint HTTP. Bila request masuk ke follower dan bersifat write operation, request diteruskan ke leader aktif. Leader kemudian mengeksekusi perubahan state dan, untuk komponen tertentu, mengirim invalidasi atau replikasi ke peer. Pola ini menjaga agar satu node menjadi sumber keputusan utama untuk operasi mutasi, sementara follower tetap dapat melayani request baca atau meneruskan request sesuai kebutuhan.



Gambar 3.1 Leader Election (.env)



Gambar 3.2 Contoh Pergantian Leader pada Startup Berurutan

3.2. Alur Konsensus Raft

Raft dipakai secara pragmatis untuk tiga fungsi utama: memilih leader, menjaga heartbeat, dan memastikan node lain mengetahui leader aktif. Setiap node memulai dalam status follower, menunggu heartbeat, lalu memulai election bila timeout terlewati.

Kandidat mengirim vote request ke peer dan akan menjadi leader jika memperoleh mayoritas suara.

Setelah terpilih, leader mengirim heartbeat berkala agar follower tidak memulai election baru. Saat follower menerima request yang seharusnya ditangani leader, follower meneruskan request ke leader aktif. Alur ini terlihat pada log pengujian yang menunjukkan pesan seperti start election, vote granted, become leader, dan heartbeat from leader.

3.3. Desain Lock, Queue dan Cache

Distributed lock manager menyimpan informasi kepemilikan lock per resource. Jika resource belum terkunci, lock diberikan kepada node peminta. Jika resource sudah terkunci, request dapat ditolak atau dimasukkan ke antrean tunggu. Struktur ini memungkinkan shared lock untuk beberapa owner dan exclusive lock untuk satu owner sesuai mode yang diminta.

Distributed queue memakai consistent hashing untuk menentukan owner logis sebuah item. Data queue disimpan di SQLite agar tetap ada saat proses berjalan ulang. Selain itu, metadata pending peer dan replicated count dipakai untuk melacak status replikasi. Dengan pendekatan ini, queue menjadi lebih tahan terhadap restart lokal, meski semantik kegagalannya masih best-effort.

Distributed cache memakai state bergaya MESI: Modified, Exclusive, Shared, dan Invalid. Penulisan lokal oleh leader akan memicu invalidasi ke node lain, sedangkan pembacaan memanfaatkan cache lokal bila state masih valid. Di sisi internal, cache juga menerapkan kebijakan LRU agar entry yang paling lama tidak dipakai dapat dikeluarkan ketika kapasitas penuh.

3.4. Skema Deployment (Manual & Docker)

Deployment manual dilakukan dengan menjalankan tiga proses Python terpisah, masing-masing pada port unik, misalnya 8001, 8002, dan 8003. Konfigurasi dapat dibaca dari `.env` atau dari argumen CLI seperti `--id`, `--port`, dan `--peers`. Pendekatan ini cocok untuk debug dan inspeksi log detail.

Deployment Docker memakai Docker Compose untuk membangun tiga container node sekaligus. Mode ini lebih stabil untuk demo karena seluruh dependency, jaringan, dan port mapping dikendalikan secara konsisten. Selain itu, konfigurasi region dan latency profile dapat dipakai untuk simulasi geo-distributed behavior.

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> docker compose up --build
#16 DONE 0.1s
[+] up 7/7
 ✓ Image distributed-sync-system-node1       Built                                   5.1s
 ✓ Image distributed-sync-system-node2       Built                                   5.1s
 ✓ Image distributed-sync-system-node3       Built                                   5.1s
 ✓ Network distributed-sync-system_default   Created                                 0.1s
 ✓ Container distributed-sync-system-node3-1 Created                                 0.2s
 ✓ Container distributed-sync-system-node1-1 Created                                 0.2s
 ✓ Container distributed-sync-system-node2-1 Created                                 0.2s
Attaching to node1-1, node2-1, node3-1
node1-1  | Node node1 running at 0.0.0.0:8001

node3-1  | Node node3 running at 0.0.0.0:8003



node1-1  | [node1] Raft startedew Config   w Enable Watch   d Detach
v View in Docker Desktop   o View Config   w Enable Watch   d Detach

node3-1  | [node3] Raft startedew Config   w Enable Watch   d Detach
node2-1  | [node2] Raft started
node1-1  | ======== Running on http://0.0.0.0:8001 ========
v View in Docker Desktop   o View Config   w Enable Watch   d Detach

node3-1  | ======== Running on http://0.0.0.0:8003 ======== d Detach




node2-1  | (Press CTRL+C to quit) Config   w Enable Watch   d Detach
node3-1  | (Press CTRL+C to quit)
node3-1  | [node3] Start election (term 1)
node1-1  | [node1] Start election (term 1)
node2-1  | [node2] Vote granted to node1 (term 1)



node1-1  | [node1] Became LEADER
node2-1  | [node2] Heartbeat from leader node1 (term 1)
node3-1  | [node3] Heartbeat from leader node1 (term 1)
node2-1  | [node2] Heartbeat from leader node1 (term 1)
node3-1  | [node3] Heartbeat from leader node1 (term 1)
node3-1  | [node3] Heartbeat from leader node1 (term 1)
node2-1  | [node2] Heartbeat from leader node1 (term 1)
```

Gambar 3.3 Contoh ‘docker compose up’ saat deployment

```
PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> docker compose ps
NAME                              IMAGE                          COMMAND                  SERVICE   CREATED              STATUS              PORTS
distributed-sync-system-node1-1   distributed-sync-system-node1  "python main.py --id…"   node1     About a minute ago   Up About a minute   0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
distributed-sync-system-node2-1   distributed-sync-system-node2  "python main.py --id…"   node2     About a minute ago   Up About a minute   0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
distributed-sync-system-node3-1   distributed-sync-system-node3  "python main.py --id…"   node3     About a minute ago   Up About a minute   0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp
```

Gambar 3.4 Daftar container berjalan (docker ps)

Feature B (demo geo-distributed):

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://localhost:8001/cache/set
-H "Content-Type: application/json" -d '{\"key\":\"geo:key\",\"value\":{\"region\":\"asia\"},\"node_id\":\"node1\"
}'
{"status": "ok", "action": "set", "key": "geo:key", "value": {"region": "asia"}, "version": 1, "updated_by": "node
1", "state": "leader", "region": "asia", "leader_id": "node1", "term": 1}
```

Gambar 3.5 Feature B: Update Cache pada Leader

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://localhost:8002/cache/status
{"status": "ok", "cache": {"geo:key": {"value": {"region": "asia"}, "version": 1, "updated_by": "node1", "state": "
M", "last_access": 1777372810.268275}}, "cache_stats": {"hits": 1, "misses": 0, "size": 1}, "region": "asia", "lead
er_id": "node1", "term": 1, "state": "leader"}
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://localhost:8003/cache/status
{"status": "ok", "cache": {"geo:key": {"value": {"region": "asia"}, "version": 1, "updated_by": "node1", "state": "
M", "last_access": 1777372810.268275}}, "cache_stats": {"hits": 1, "misses": 0, "size": 1}, "region": "asia", "lead
er_id": "node1", "term": 1, "state": "leader"}
```

Gambar 3.6 Feature B: Cek Follower setelah Update

3.5. Failure Detection dan Metrics

Selain Raft, sistem juga memiliki failure detector sederhana untuk memantau heartbeat dan kontak terakhir dengan peer. Komponen ini dipakai untuk menandai peer yang tidak responsif sebagai suspected sehingga health check bisa memberi gambaran kesehatan cluster secara lebih jelas.

Di sisi observability, metrics collector menyimpan counter, timing, dan gauge runtime. Endpoint `/metrics` menampilkan ringkasan seperti jumlah request, state Raft, leader aktif, ukuran cache dan queue, serta daftar peer yang sedang dicurigai. Dengan begitu, pengujian tidak hanya bergantung pada log, tetapi juga pada output terstruktur yang mudah dibaca saat demo.

BAB IV
IMPLEMENTASI TIAP KOMPONEN

4.1. Implementasi Lock Manager

Lock manager menyimpan peta lock dan daftar waiting queue per resource. Saat request acquire masuk, sistem mengecek apakah resource masih kosong, masih dipegang secara shared, atau sedang dikunci secara exclusive. Pada kondisi resource kosong, lock langsung diberikan. Pada kondisi shared, request shared lain dapat bergabung sebagai owner, sedangkan request exclusive akan menunggu.

Perintah uji komponen lock :

1. Acquire lock pertama (node1 jadi owner awal).

```
curl.exe --% -X POST http://localhost:8001/lock/acquire -H
"Content-Type: application/json" -d
"{\"resource\":\"demo-lock\",\"node_id\":\"node1\"}"
```

2. Uji kontensi dari follower (node2 mencoba lock resource yang sama, lalu diputuskan terpusat oleh leader).

```
curl.exe --% -X POST http://localhost:8002/lock/acquire -H
"Content-Type: application/json" -d
"{\"resource\":\"demo-lock\",\"node_id\":\"node2\"}"
```

3. Release lock dari owner aktif (node1 melepas lock agar resource tersedia lagi).

```
curl.exe --% -X POST http://localhost:8001/lock/release -H
"Content-Type: application/json" -d
"{\"resource\":\"demo-lock\",\"node_id\":\"node1\"}"
```

4. Cek status lock saat ini (validasi owner, queue tunggu, leader_id, dan state node).

```
curl.exe -X GET http://localhost:8001/lock/status
```

5. Cek apakah ada deadlock terdeteksi (membaca wait-for cycle yang aktif).\

```
curl.exe -X GET http://localhost:8001/admin/locks/deadlocks
```

6. Resolusi deadlock otomatis (pilih victim dengan strategi `youngest`)

```
curl.exe --% -X POST
http://localhost:8001/admin/locks/resolve -H "Content-Type:
application/json" -d "{\"strategy\":\"youngest\"}"
```

Pada kebutuhan akademik dan debugging, lock manager juga menyediakan kemampuan deteksi deadlock berbasis wait-for graph. Graf dibangun dari hubungan node yang menunggu resource yang sedang dipegang node lain. Bila terdapat siklus, sistem

menganggap ada deadlock dan dapat memilih victim untuk diresolusi. Strategi resolusi mendukung pemilihan youngest, oldest, atau random agar hasilnya mudah didemokan.

Pada layer HTTP, endpoint `/lock/acquire`, `/lock/release`, `/lock/status`, `/admin/locks/deadlocks`, dan `/admin/locks/resolve` memberikan akses langsung untuk pengujian. Jika request masuk ke follower, request diteruskan ke leader terlebih dahulu agar keputusan lock tetap terpusat.

4.2. Implementasi Distributed Queue

Queue manager menggunakan consistent hashing untuk menentukan owner sebuah item. Setiap node dan peer dimasukkan ke ring hash, lalu item dipetakan ke node yang bertanggung jawab. Ketika item masuk ke owner yang benar, data disimpan ke SQLite dengan metadata node pengirim, jumlah replika yang sudah diterima, dan daftar peer yang masih menunggu ack.

Perintah uji komponen queue:

1. Enqueue dari follower (uji forwarding ke leader/owner queue).

```
curl.exe --% -X POST http://localhost:8002/queue/enqueue -H
"Content-Type: application/json" -d
"{\"item\":\"job-1\",\"node_id\":\"node1\"}"
```

2. Cek status queue cluster (lihat ukuran queue, pending replication, dan metadata item).

```
curl.exe -X GET http://localhost:8001/queue/status
```

3. Dequeue item paling depan (verifikasi FIFO + sinkronisasi state runtime dan SQLite).

```
curl.exe --% -X POST http://localhost:8001/queue/dequeue -H
"Content-Type: application/json" -d
"{\"node_id\":\"node1\"}"
```

4. Enqueue item kedua dengan requester berbeda (uji konsistensi metadata node_id pengirim).

```
curl.exe --% -X POST http://localhost:8002/queue/enqueue -H
"Content-Type: application/json" -d
"{\"item\":\"job-2\",\"node_id\":\"node2\"}"
```

5. Cek status dari node lain (validasi visibilitas state queue antar node).

```
curl.exe -X GET http://localhost:8003/queue/status
```

Jika request enqueue datang ke node yang bukan owner, request dapat diteruskan ke owner tersebut. Setelah item berhasil disimpan, leader memulai proses replikasi asinkron ke peer lain dengan retry. Pola ini cukup untuk menunjukkan konsep durability dan distribusi data, walaupun belum menerapkan consensus commit penuh.

Status queue dapat dilihat melalui endpoint `/queue/status`, sedangkan replikasi menerima payload internal melalui `/queue/replicate`. Implementasi dequeue menghapus item paling depan dari antrian dan menghapus record terkait dari SQLite agar state runtime dan state persistent tetap sinkron.

4.3. Implementasi Cache Coherence

Cache manager menyimpan entry dalam struktur OrderedDict agar urutan akses dapat dipakai untuk LRU. Setiap entry memiliki value, version, updated_by, state, dan last_access. Ketika leader menulis ke cache, state lokal ditandai sebagai Modified dan sistem mengirim invalidasi ke node lain. Node follower yang menerima invalidasi akan menandai entry menjadi Invalid tanpa langsung menghapus metadata lama.

Perintah uji komponen cache:

1. Set nilai cache pada leader (menguji write path + update version/state entry).

```
curl.exe --% -X POST http://localhost:8001/cache/set -H
"Content-Type: application/json" -d
"{\"key\":\"user:1\",\"value\":{\"name\":\"naufal\"},\"node_
id\":\"node1\"}"
```

2. Get dari follower (menguji apakah nilai terlihat lintas node setelah update leader).

```
curl.exe -X GET "http://localhost:8002/cache/get?key=user:1"
```

3. Cek status cache node lain (validasi statistik cache dan state coherence).

```
curl.exe -X GET http://localhost:8003/cache/status
```

4. Delete key pada leader (menguji invalidasi/hapus data terkoordinasi).

```
curl.exe --% -X POST http://localhost:8001/cache/delete -H
"Content-Type: application/json" -d
"{\"key\":\"user:1\",\"node_id\":\"node1\"}"
```

5. Get ulang dari follower (verifikasi key sudah tidak valid/terhapus).

```
curl.exe -X GET "http://localhost:8002/cache/get?key=user:1"
```

Desain ini sengaja dibuat agar sederhana tetapi tetap representatif: read yang masih valid dapat dilayani cepat dari cache lokal, sedangkan write dari leader menjaga konsistensi melalui invalidasi. Endpoint `/cache/set`, `/cache/get`, `/cache/delete`, `/cache/invalidate`, dan `/cache/status` mendukung skenario demo serta pembuktian koherensi data antar node.

4.4. API dan OpenAPI/Swagger

Seluruh endpoint utama didokumentasikan melalui OpenAPI 3.0.3. Dokumentasi ini mencakup health check, handler internal Raft, lock, queue, cache, serta endpoint administrasi deadlock. Dengan dokumentasi tersebut, client dapat memahami struktur request body, response code, dan fungsi tiap endpoint tanpa harus membaca source code secara langsung.

Swagger UI dapat dipakai untuk eksplorasi API secara interaktif, sedangkan file OpenAPI machine-readable memudahkan integrasi tooling atau validasi otomatis. Keberadaan dokumentasi ini sangat penting dalam konteks presentasi akademik karena menunjukkan bahwa sistem tidak hanya berfungsi, tetapi juga terdokumentasi dengan baik.

4.5. Failure Detector dan Metrics

Failure detector bekerja berdampingan dengan Raft untuk mencatat heartbeat yang masuk dan kontak yang berhasil dilakukan ke peer. Saat node tidak merespons dalam interval tertentu, peer tersebut dimasukkan ke daftar suspected agar health check dan log operasional lebih informatif.

Metrics collector dipakai untuk merekam request count, error count, state Raft, ukuran cache, ukuran queue, dan indikator operasional lain. Tambahan ini membuat sistem lebih mudah dievaluasi selama percobaan manual maupun Docker karena status runtime bisa dibaca dari endpoint khusus tanpa harus menelusuri log mentah.

BAB V
SKENARIO UJI DAN HASIL FUNGSIONAL

5.1. Skenario Uji Lock

Pengujian lock memverifikasi apakah request acquire, release, status, dan deteksi deadlock berjalan konsisten pada tiga node. Skenario utamanya adalah memastikan lock yang diminta node follower tetap diteruskan ke leader, lalu kepemilikan lock dan status owner yang ditampilkan kembali sesuai dengan operasi yang dilakukan. Dari percobaan manual, lock dapat diambil oleh node peminta, lalu dilepas kembali dengan benar. Status lock menunjukkan owner aktif dan leader_id yang sedang memimpin. Pada pengujian unit, deadlock antara dua node juga berhasil dideteksi, lalu diselesaikan dengan strategi victim selection.

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://loc
alhost:8001/lock/acquire -H "Content-Type: application/json" -d '{\"resource\":\"demo-lock\"
,\"node_id\":\"node1\"}'
{"status": "ok", "action": "acquire", "resource": "demo-lock", "owner": "node1", "leader_id"
: "node3", "term": 1, "state": "leader"}
```

Gambar 5.1 Contoh ‘acquire_lock’ pada Leader (Output)

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://loca
lhost:8003/lock/status
{"status": "ok", "locks": {"demo-lock": "node1"}, "leader_id": "node3", "term": 1, "state":
"leader"}
```

Gambar 5.2 Status Lock di Endpoint ‘lock_status’

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://loc
alhost:8001/lock/release -H "Content-Type: application/json" -d '{\"resource\":\"demo-lock\"
,\"node_id\":\"node1\"}'
{"status": "ok", "action": "release", "resource": "demo-lock", "owner": "node1", "leader_id"
: "node3", "term": 1, "state": "leader"}
```

Gambar 5.3 Proses ‘release_lock’ pada Leader

5.2. Skenario Uji Queue

Queue diuji untuk melihat apakah item yang masuk bisa disimpan, dibaca kembali, dan dikeluarkan secara konsisten pada lingkungan multi-node. Skenario ini juga menguji apakah request dari follower diteruskan ke leader dengan benar dan apakah metadata replikasi tetap sinkron setelah enqueue atau dequeue. Hasil pengujian menunjukkan bahwa request dari follower dapat diteruskan ke leader, item dapat disimpan ke queue, dan status queue dapat dibaca kembali. Bukti tambahan dari unit test memastikan bahwa metadata pending peers dan ack replika berubah sesuai ekspektasi.

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://loc
alhost:8002/queue/enqueue -H "Content-Type: application/json" -d '{\"item\":\"job-1\",\"node
_id\":\"node1\"}'
{"status": "ok", "action": "enqueue", "item": "job-1", "requested_by": "node1", "queue_size"
: 3, "leader_id": "node3", "term": 1, "state": "leader"}
```

Gambar 5.4 Enqueue Item pada Queue (Node)

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://loca
lhost:8001/queue/status
{"status": "ok", "queue": [{"id": 1, "item": "job-1", "node_id": "node2"}, {"id": 2, "item":
 "job-1", "node_id": "node1", "replicated_count": 2, "pending_peers": ""}, {"id": 3, "item":
 "job-1", "node_id": "node1", "replicated_count": 2, "pending_peers": ""}], "queue_size": 3,
 "leader_id": "node3", "term": 1, "state": "leader"}
```

Gambar 5.5 Status Queue dan Antrian Pending

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://loc
alhost:8002/queue/dequeue -H "Content-Type: application/json" -d '{\"node_id\":\"node1\"}'
{"status": "ok", "action": "dequeue", "item": "job-1", "enqueued_by": "node2", "requested_by
": "node1", "queue_size": 2, "leader_id": "node3", "term": 1, "state": "leader"}
```

Gambar 5.6 Dequeue dan Konsistensi Data antar Node

5.3. Skenario Uji Cache

Cache diuji untuk memverifikasi alur set, get, delete, invalidate, dan status pada skema coherence antar node. Fokusnya adalah memastikan perubahan dari leader dapat terlihat pada follower, lalu invalidasi dan pembaruan state cache berjalan sesuai model MESI-like yang dipakai sistem. Dari percobaan manual, key yang di-set pada leader dapat dibaca kembali, lalu dihapus atau di-invalidate pada node lain sesuai alur replikasi. Pada pengujian unit, skenario MESI-like menunjukkan bahwa invalidasi pada peer membuat entry tidak lagi dianggap valid, dan LRU bekerja saat kapasitas cache dibatasi.

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://loc
alhost:8001/cache/set -H "Content-Type: application/json" -d '{\"key\":\"user:1\",\"value\":
{\"name\":\"naufal\"},\"node_id\":\"node1\"}'
{"status": "ok", "action": "set", "key": "user:1", "value": {"name": "naufal"}, "version": 1
, "updated_by": "node1", "state": "leader", "region": "asia", "leader_id": "node3", "term":
1}
```

Gambar 5.7 Uji Cache Coherence antar Node

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET "http://loc
alhost:8002/cache/get?key=user:1"
{"status": "ok", "action": "get", "key": "user:1", "value": {"name": "naufal"}, "version": 1
, "updated_by": "node1", "leader_id": "node3", "term": 1, "state": "leader"}
```

Gambar 5.8 Contoh Cek Nilai ‘user-1’ setelah Update

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://loca
lhost:8003/cache/status
{"status": "ok", "cache": {"user:1": {"value": {"name": "naufal"}, "version": 1, "updated_by
": "node1", "state": "M", "last_access": 1777369215.3949087}}, "cache_stats": {"hits": 2, "m
isses": 1, "size": 1}, "region": "asia", "leader_id": "node3", "term": 1, "state": "leader"}
```

Gambar 5.9 Status Cache (Statistik dan Isi)

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X POST http://loc
alhost:8001/cache/delete -H "Content-Type: application/json" -d '{\"key\":\"user:1\",\"node_
id\":\"node1\"}'
{"status": "ok", "action": "delete", "key": "user:1", "requested_by": "node1", "region": "as
ia", "leader_id": "node3", "term": 1, "state": "leader"}
```

Gambar 5.10 Hapus Entri Cache (cache_delete)

5.4. Uji Integrasi 3 Node

Uji integrasi tiga node memverifikasi bahwa seluruh komponen inti dapat berjalan bersama dalam satu cluster, bukan hanya lolos secara terpisah di unit test. Skenario ini dipakai untuk melihat apakah follower dapat meneruskan request ke leader, leader dapat melakukan mutasi, dan invalidasi/replikasi dapat menjangkau node lain secara konsisten. Pengujian ini penting karena membuktikan bahwa komponen yang secara individual lulus unit test juga dapat berinteraksi dalam skenario sistem yang lebih realistis.

5.5. Perbandingan Mode Uji

Hasil pengujian terakhir menunjukkan perbedaan yang jelas antar mode. Pada mode manual dengan `.env`, cluster dapat berjalan dan leader election terbentuk, tetapi queue menyimpan state SQLite antar run sehingga hasil awal tidak selalu bersih. Pada mode manual tanpa `.env`, election lebih tidak stabil dan leader dapat berubah-ubah, sehingga beberapa request forwarding menjadi gagal atau tidak konsisten.

Mode Docker memberi hasil paling stabil karena jaringan, environment variable, dan port mapping dikelola konsisten oleh Compose. Di mode ini leader terbentuk dengan jelas, follower tetap mengikuti heartbeat, dan operasi lock, queue, cache, serta metrics berjalan sesuai ekspektasi. Untuk konfigurasi geo-distributed, container juga menunjukkan pemisahan region dan latency profile yang berbeda, sehingga simulasi antar-region dapat dibuktikan secara eksplisit.

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> python -m unittest discover -s tests/unit -p "test
_*.py"
C:\Python313\Lib\asyncio\proactor_events.py:725: ResourceWarning: unclosed database in <sqlite3.Connection object a
t 0x000001B1B43B6C50>
  async def sock_connect(self, sock, address):
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Python313\Lib\asyncio\proactor_events.py:725: ResourceWarning: unclosed database in <sqlite3.Connection object a
t 0x000001B1B43B6D40>
  async def sock_connect(self, sock, address):
ResourceWarning: Enable tracemalloc to get the object allocation traceback
...........<attrs generated methods aiohttp.web_routedef.RouteDef>:19: ResourceWarning: unclosed database in <sqlit
e3.Connection object at 0x000001B1B43B6D40>
  _setattr = _cached_setattr_get(self)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
<attrs generated methods aiohttp.web_routedef.RouteDef>:19: ResourceWarning: unclosed database in <sqlite3.Connecti
on object at 0x000001B1B43B67A0>
  _setattr = _cached_setattr_get(self)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
<attrs generated methods aiohttp.web_routedef.RouteDef>:19: ResourceWarning: unclosed database in <sqlite3.Connecti
on object at 0x000001B1B43B7D30>
  _setattr = _cached_setattr_get(self)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
<attrs generated methods aiohttp.web_routedef.RouteDef>:19: ResourceWarning: unclosed database in <sqlite3.Connecti
on object at 0x000001B1B549C130>
  _setattr = _cached_setattr_get(self)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
<attrs generated methods aiohttp.web_routedef.RouteDef>:19: ResourceWarning: unclosed database in <sqlite3.Connecti
on object at 0x000001B1B549C5E0>
  _setattr = _cached_setattr_get(self)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
<attrs generated methods aiohttp.web_routedef.RouteDef>:19: ResourceWarning: unclosed database in <sqlite3.Connecti
on object at 0x000001B1B549C6D0>
  _setattr = _cached_setattr_get(self)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
.......
----------------------------------------------------------------------
Ran 19 tests in 0.379s

OK
```

Gambar 5.11 Menjalankan Suite Test seluruh Modul

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> python benchmarks/load_test_scenarios.py
Benchmark results
- lock_acquire_release: avg=0.0007 ms, p95=0.0007 ms, min=0.0006 ms, max=0.0126 ms
- queue_enqueue_dequeue: avg=8.6981 ms, p95=10.4188 ms, min=7.455 ms, max=20.3537 ms
- cache_set_get_invalidate: avg=0.0016 ms, p95=0.0017 ms, min=0.0015 ms, max=0.0372 ms
Saved CSV to D:\Semester 6\Sister\Tugas 3\distributed-sync-system\benchmarks\results.csv
```

Gambar 5.12 Output benchmark run

```
benchmarks > results.csv > data
1  scenario,iterations,avg_ms,p95_ms,min_ms,max_ms
2  lock_acquire_release,5000,0.0007,0.0007,0.0006,0.0126
3  queue_enqueue_dequeue,2000,8.6981,10.4188,7.455,20.3537
4  cache_set_get_invalidate,5000,0.0016,0.0017,0.0015,0.037
```

Gambar 5.13 Hasil Benchmark Disimpan ke CSV

```
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://localhost:8001/health
{"node": "node1"}
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://localhost:8002/health
{"node": "node2"}
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://localhost:8003/health
{"node": "node3"}
(.venv) PS D:\Semester 6\Sister\Tugas 3\distributed-sync-system> curl.exe -X GET http://localhost:8001/lock/status
{"status": "ok", "locks": {}, "leader_id": "node1", "term": 1, "state": "leader"}
```

Gambar 5.14 Smoke Test Cepat

BAB VI
ANALISIS PERFORMA DAN TRADE-OFF

6.1. Metode Benchmark

Benchmark dijalankan menggunakan skrip `benchmarks/load_test_scenarios.py` dengan tiga skenario utama: lock acquire/release, queue enqueue/dequeue, dan cache set/get/invalidate. Hasil mentah disimpan pada `benchmarks/results.csv` sehingga pengukuran dapat ditinjau ulang dan dibandingkan dengan run berikutnya.

6.2. Hasil Angka (rata-rata, p95, max)

Table 6.1 Hasil Output Benchmark

Scenario	Iteration	Average (ms)	P95 (ms)	Min (ms)	Max (ms)
lock_acquire_release	5000	0.0007	0.0007	0.0006	0.0126
queue_enqueue_dequeue	2000	8.6981	10.4188	7.4550	20.3537
cache_set_get_invalidate	5000	0.0016	0.0017	0.0015	0.0372

6.3. Interpretasi

Data di atas menunjukkan bahwa lock dan cache memiliki latensi sangat rendah karena mayoritas operasi berlangsung di memori. Queue lebih lambat karena setiap enqueue dan dequeue melibatkan SQLite, pengelolaan metadata replikasi, serta potensi forwarding antar node. Perbedaan ini wajar karena queue sengaja didesain lebih tahan simpan dibanding lock dan cache.

P95 yang relatif dekat dengan rata-rata menandakan sistem cukup stabil pada jalur lokal. Nilai maksimum yang lebih tinggi pada queue menunjukkan adanya outlier kecil yang dapat berasal dari proses sinkronisasi storage, overhead thread, atau variasi scheduler sistem operasi. Secara keseluruhan, hasil benchmark mendukung kesimpulan bahwa sistem cocok untuk demo akademik dengan karakteristik performa yang transparan dan masuk akal.

6.4. Dampak Konsistensi terhadap Throughput/Latency

Semakin kuat konsistensi yang dituntut, semakin besar biaya sinkronisasi yang dibayar sistem. Lock harus melewati leader, queue harus mempertahankan metadata persistensi dan replikasi, dan cache harus mengirim invalidasi saat ada write. Akibatnya, throughput operasi tulis tentu lebih rendah daripada implementasi in-memory murni. Namun trade-off ini memang diperlukan agar state antar node tidak divergen.

BAB VII
KESIMPULAN

Proyek distributed-sync-system berhasil menunjukkan implementasi inti sinkronisasi terdistribusi dalam bentuk yang dapat didemokan, diuji, dan dijelaskan secara akademik. Sistem telah memiliki leader election, distributed lock manager, distributed queue dengan persistensi, cache coherence dengan invalidasi, failure detector, metrics endpoint, dokumentasi API, serta dukungan deployment manual dan Docker.

Secara keseluruhan, hasil pengujian membuktikan bahwa rancangan arsitektur tiga node berjalan sesuai tujuan. Walaupun masih ada batasan pada model konsensus dan replikasi, implementasi ini sudah cukup kuat sebagai bukti pemahaman terhadap konsep koordinasi node, konsistensi data, dan trade-off performa dalam sistem terdistribusi.

LAMPIRAN

Link Github : distributed-sync-system

Link Video : Distributed Sync System | Tugas 3 Sistem Paralel dan Terdistribusi